
Chapter 4: Divide-and-Conquer

Chapter 4 overview

Recall the divide-and-conquer paradigm, which we used for merge sort:

Divide the problem into one or more subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.

Base case: If the subproblems are small enough, just solve them by brute force.

Combine the subproblem solutions to form a solution to the original problem.

We look at two algorithms for multiplying square matrices, based on divide-and-conquer.

Analyzing divide-and-conquer algorithms

Use a recurrence to characterize the running time of a divide-and-conquer algorithm. Solving the recurrence gives us the asymptotic running time.

A **recurrence** is a function is defined in terms of

- one or more base cases, and
- itself, with smaller arguments.

A recurrence could have 0, 1, or more functions that satisfy it. **Well defined** if at least 1 function satisfies; otherwise, **ill defined**.

Algorithmic recurrences

Interested in recurrences that describe running times of algorithms.

A recurrence $T(n)$ is **algorithmic** if for every sufficiently large **threshold** constant $n_0 > 0$:

- For all $n < n_0$, $T(n) = \Theta(1)$. [Can consider the running time constant for small problem sizes.]
- For all $n \geq n_0$, every path of recursion terminates in a defined base case within a finite number of recursive invocations. [The recursive algorithm terminates.]

Conventions

Will often state recurrences without base cases. When analyzing algorithms, assume that if no base case is given, the recurrence is algorithmic. Allows us to pick any sufficiently large threshold constant n_0 without changing the asymptotic behavior of the solution.

Ceilings and floors in divide-and-conquer recurrences don't change the asymptotic solution $\Rightarrow$ often state algorithmic recurrences without floors and ceilings, even though to be precise, they should be there. *[Example: recurrence for merge sort is really $T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n)$.]*

Some recurrences are inequalities rather than equations.

Example: $T(n) \leq 2T(n/2) + \Theta(n)$ gives only an upper bound on $T(n)$, so state the solution using O -notation rather than Θ -notation.

Examples of recurrences arising from divide-and-conquer algorithms

- $n \times n$ matrix multiplication by breaking into 8 subproblems of size $n/2 \times n/2$: $T(n) = 8T(n/2) + \Theta(1)$. Solution: $T(n) = \Theta(n^3)$.
Strassen's algorithm for $n \times n$ matrix multiplication by breaking into 7 sub-
- problems of size $n/2 \times n/2$: $T(n) = 7T(n/2) + \Theta(1)$. Solution: $T(n) = \Theta(n^{\lg 7}) = O(n^{2.81})$.
- An algorithm that breaks a problem of size n into one subproblem of size $n/3$ and another of size $2n/3$, taking $\Theta(n)$ time to divide and combine: $T(n) = T(n/3) + T(2n/3) + \Theta(n)$. Solution: $T(n) = \Theta(n \lg n)$.
- An algorithm that breaks a problem of size n into one subproblem of size $n/5$ and another of size $7n/10$, taking $\Theta(n)$ time to divide and combine: $T(n) = T(n/5) + T(7n/10) + \Theta(n)$. Solution: $T(n) = \Theta(n)$. *[This is the recurrence for order-statistic algorithm in Chapter 9 that takes linear time in the worst case.]*
- Subproblems don't always have to be a constant fraction of the original problem size. **Example:** recursive linear search creates one subproblem and it has one element less than the original problem. Time to divide and combine is $\Theta(1)$, giving $T(n) = T(n-1) + \Theta(1)$. Solution: $T(n) = \Theta(n)$.

Methods for solving recurrences

The chapter contains four methods for solving recurrences. Each gives asymptotic bounds.

- Substitution method: Guess the solution, then use induction to prove that it's correct.
- Recursion-tree method: Draw out a recursion tree, determine the costs at each level, and sum them up. Useful for coming up with a guess for the substitution method.
- Master method: A cookbook method for recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a > 0$ and $b > 1$ are constants, subject to certain

conditions. Requires memorizing three cases, but applies to many divide-and-conquer algorithms.

- Akra-Bazzi method: A general method for solving divide-and-conquer recurrences. Requires calculus, but applies to recurrences beyond those solved by the master method. *[We do not cover the Akra-Bazzi method.]*

In this course, we concentrate only on two acceptable ways of solving recurrences: the substitution method and the master method.

Multiplying square matrices

Input: Three $n \times n$ (square) matrices, $A = (a_{ij})$, $B = (b_{ij})$, and $C = (c_{ij})$.

Result: The matrix product $A \cdot B$ is added into C , so that

$$c_{ij} = c_{ij} + \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

for $i, j = 1, 2, \dots, n$.

If only the product $A \cdot B$ is needed, then zero out all entries of C beforehand.

MATRIX-MULTIPLY(A, B, C, n)

```

1  for  $i = 1$  to  $n$                 // compute entries in each of  $n$  rows
2      for  $j = 1$  to  $n$             // compute  $n$  entries in row  $i$ 
3          for  $k = 1$  to  $n$ 
4               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$  // add in one more term of equation (4.1)
```

Time: $\Theta(n^3)$ because of triply nested loops.

Simple divide-and-conquer algorithm

For simplicity, assume that C is initialized to 0, so computing $C = A \cdot B$.

If $n > 1$, partition each of A, B, C into four $n/2 \times n/2$ matrices:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

Rewrite $C = A \cdot B$ as

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix},$$

giving the four equations

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21},$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22},$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21},$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}.$$

Each of these equations multiplies two $n/2 \times n/2$ matrices and then adds their $n/2 \times n/2$ products. Assume that n is an exact power of 2, so that submatrix dimensions are always integer.

Use these equations to get a divide-and-conquer algorithm:

MATRIX-MULTIPLY-RECURSIVE(A, B, C, n)

```

1  if  $n == 1$ 
2    // Base case.
3       $c_{11} = c_{11} + a_{11} \cdot b_{11}$ 
4      return
5    // Divide.
6    partition  $A, B$ , and  $C$  into  $n/2 \times n/2$  submatrices
        $A_{11}, A_{12}, A_{21}, A_{22}; B_{11}, B_{12}, B_{21}, B_{22};$ 
       and  $C_{11}, C_{12}, C_{21}, C_{22}$ ; respectively
7    // Conquer.
8    MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{11}, C_{11}, n/2$ )
9    MATRIX-MULTIPLY-RECURSIVE( $A_{11}, B_{12}, C_{12}, n/2$ )
10   MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{11}, C_{21}, n/2$ )
11   MATRIX-MULTIPLY-RECURSIVE( $A_{21}, B_{12}, C_{22}, n/2$ )
12   MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{21}, C_{11}, n/2$ )
13   MATRIX-MULTIPLY-RECURSIVE( $A_{12}, B_{22}, C_{12}, n/2$ )
14   MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{21}, C_{21}, n/2$ )
15   MATRIX-MULTIPLY-RECURSIVE( $A_{22}, B_{22}, C_{22}, n/2$ )
```

[The book briefly discusses the question of how to avoid copying entries when partitioning matrices. Can partition matrices without copying entries by instead using index calculations. Identify a submatrix by ranges of row and column matrices from the original matrix. End up representing a submatrix differently from how we represent the original matrix. The advantage of avoiding copying is that partitioning would take only constant time, instead of $\Theta(n^2)$ time. The result of the asymptotic analysis won't change, but using index calculations to avoid copying gives better constant factors.]

Analysis

Let $T(n)$ be the time to multiply two $n \times n$ matrices.

Base case: $n = 1$. Perform one scalar multiplication: $\Theta(1)$.

Recursive case: $n > 1$.

- Dividing takes $\Theta(1)$ time, using index calculations. [Otherwise, $\Theta(n^2)$ time.]
- Conquering makes 8 recursive calls, each multiplying $n/2 \times n/2$ matrices $\Rightarrow 8T(n/2)$.
- No combine step, because C is updated in place.

Recurrence (omitting the base case) is $T(n) = 8T(n/2) + \Theta(1)$. Can use master method to show that it has solution $T(n) = \Theta(n^3)$.

Asymptotically, no better than the obvious method.

Bushiness of recursion trees: Compare this recurrence with the merge-sort recurrence $T(n) = 2T(n/2) + \Theta(n)$. If we draw out the recursion trees, the factor of 2 in the merge-sort recurrence says that each non-leaf node has 2 children. But the factor of 8 in the recurrence for MATRIX-MULTIPLY-RECURSIVE says that each non-leaf node has 8 children. Get a bushier tree with many more leaves, even though internal nodes have a smaller cost.

Strassen's algorithm

Idea: Make the recursion tree less bushy. Perform only 7 recursive multiplications of $n/2 \times n/2$ matrices, rather than 8. Will cost several additions/subtractions of $n/2 \times n/2$ matrices.

Since a subtraction is a “negative addition,” just refer to all additions and subtractions as additions.

Example of reducing multiplications: Given x and y , compute $x^2 - y^2$. Obvious way uses 2 multiplications and one subtraction. But observe:

$$x^2 - y^2 = (x + y)(x - y),$$

so at the expense of one extra addition, can get by with only 1 multiplication. Not a big deal if x, y are scalars, but can make a difference if they are matrices.

The algorithm:

1. Same base case as before, when $n = 1$.
2. When $n > 1$, then as in the recursive method, partition each of the matrices into four $n/2 \times n/2$ submatrices. Time: $\Theta(1)$, using index calculations.
3. Create 10 matrices $S_1, S_2, \dots, S_{10}$. Each is $n/2 \times n/2$ and is the sum or difference of two matrices created in previous step. Time: $\Theta(n^2)$ to create all 10 matrices.
4. Create and zero the entries of 7 matrices $P_1, P_2, \dots, P_7$, each $n/2 \times n/2$. Time: $\Theta(n^2)$.
5. Using the submatrices of A and B and the matrices $S_1, S_2, \dots, S_{10}$, recursively compute $P_1, P_2, \dots, P_7$. Time: $7T(n/2)$.
6. Update the four $n/2 \times n/2$ submatrices $C_{11}, C_{12}, C_{21}, C_{22}$ of C by adding and subtracting various combinations of the P_i . Time: $\Theta(n^2)$.

Analysis

Recurrence will be $T(n) = 7T(n/2) + \Theta(n^2)$. By the master method, solution is $T(n) = \Theta(n^{\lg 7})$. Since $\lg 7 < 2.81$, the running time is $O(n^{2.81})$, beating the $\Theta(n^3)$ -time methods.

Details

Step 2: Create the 10 matrices

$$\begin{aligned} S_1 &= B_{12} - B_{22} , \\ S_2 &= A_{11} + A_{12} , \\ S_3 &= A_{21} + A_{22} , \\ S_4 &= B_{21} - B_{11} , \\ S_5 &= A_{11} + A_{22} , \\ S_6 &= B_{11} + B_{22} , \\ S_7 &= A_{12} - A_{22} , \\ S_8 &= B_{21} + B_{22} , \\ S_9 &= A_{11} - A_{21} , \\ S_{10} &= B_{11} + B_{12} . \end{aligned}$$

Add or subtract $n/2 \times n/2$ matrices 10 times $\Rightarrow$ time is $\Theta(n^2)$.

Step 4: Compute the 7 matrices

$$\begin{aligned} P_1 &= A_{11} \cdot S_1 = A_{11} \cdot B_{12} - A_{11} \cdot B_{22} , \\ P_2 &= S_2 \cdot B_{22} = A_{11} \cdot B_{22} + A_{12} \cdot B_{22} , \\ P_3 &= S_3 \cdot B_{11} = A_{21} \cdot B_{11} + A_{22} \cdot B_{11} , \\ P_4 &= A_{22} \cdot S_4 = A_{22} \cdot B_{21} - A_{22} \cdot B_{11} , \\ P_5 &= S_5 \cdot S_6 = A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} , \\ P_6 &= S_7 \cdot S_8 = A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22} , \\ P_7 &= S_9 \cdot S_{10} = A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12} . \end{aligned}$$

The only multiplications needed are in the middle column; right-hand column just shows the products in terms of the original submatrices of A and B .

Step 5: Add and subtract the P_i to construct submatrices of C :

$$\begin{aligned} C_{11} &= P_5 + P_4 - P_2 + P_6 , \\ C_{12} &= P_1 + P_2 , \\ C_{21} &= P_3 + P_4 , \\ C_{22} &= P_5 + P_1 - P_3 - P_7 . \end{aligned}$$

To see how these computations work, expand each right-hand side, replacing each P_i with the submatrices of A and B that form it, and cancel terms: [We expand out all four right-hand sides here.]

$$\begin{array}{r}
A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} \\
\quad - A_{22} \cdot B_{11} \quad \quad \quad + A_{22} \cdot B_{21} \\
\quad - A_{11} \cdot B_{22} \quad \quad \quad - A_{12} \cdot B_{22} \\
\quad \quad \quad - A_{22} \cdot B_{22} - A_{22} \cdot B_{21} + A_{12} \cdot B_{22} + A_{12} \cdot B_{21} \\
\hline
A_{11} \cdot B_{11} \quad \quad \quad + A_{12} \cdot B_{21}
\end{array}$$

$$\begin{array}{r}
A_{11} \cdot B_{12} - A_{11} \cdot B_{22} \\
\quad + A_{11} \cdot B_{22} + A_{12} \cdot B_{22} \\
\hline
A_{11} \cdot B_{12} \quad \quad \quad + A_{12} \cdot B_{22}
\end{array}$$

$$\begin{array}{r}
A_{21} \cdot B_{11} + A_{22} \cdot B_{11} \\
\quad - A_{22} \cdot B_{11} + A_{22} \cdot B_{21} \\
\hline
A_{21} \cdot B_{11} \quad \quad \quad + A_{22} \cdot B_{21}
\end{array}$$

$$\begin{array}{r}
A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} \\
\quad - A_{11} \cdot B_{22} \quad \quad \quad + A_{11} \cdot B_{12} \\
\quad \quad \quad - A_{22} \cdot B_{11} \quad \quad \quad - A_{21} \cdot B_{11} \\
- A_{11} \cdot B_{11} \quad \quad \quad - A_{11} \cdot B_{12} + A_{21} \cdot B_{11} + A_{21} \cdot B_{12} \\
\hline
\quad \quad \quad A_{22} \cdot B_{22} \quad \quad \quad + A_{21} \cdot B_{12}
\end{array}$$

Theoretical and practical notes

Strassen's algorithm was the first to beat $\Theta(n^3)$ time, but it's not the asymptotically fastest known. A method by Coppersmith and Winograd runs in $O(n^{2.376})$ time. Current best asymptotic bound (not practical) is $O(n^{2.37286})$.

Practical issues against Strassen's algorithm:

- Higher constant factor than the obvious $\Theta(n^3)$ -time method.
- Not good for sparse matrices.
- Not numerically stable: larger errors accumulate than in the obvious method.
- Submatrices consume space, especially if copying.

Numerical stability problem is not as bad as previously thought. And can use index calculations to reduce space requirement.

Substitution method

1. Guess the solution.

2. Use induction to find the constants and show that the solution works.

Usually use the substitution method to establish either an upper bound (O -bound) or a lower bound (Ω -bound).

Example

Determine an asymptotic upper bound on $T(n) = 2T(\lfloor n/2 \rfloor) + \Theta(n)$. Similar to the merge-sort recurrence except for the floor function. (Ensures that $T(n)$ is defined over integers.)

Guess same asymptotic upper bound as merge-sort recurrence: $T(n) = O(n \lg n)$.

Inductive hypothesis: $T(n) \leq cn \lg n$ for all $n \geq n_0$. Will choose constants

$c, n_0 > 0$ later, once we know their constraints.

Inductive step: Assume that $T(n) \leq cn \lg n$ for all numbers $\geq n_0$ and $< n$. If $n \geq 2n_0$, holds for $\lfloor n/2 \rfloor \Rightarrow T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor$. Substitute into the recurrence:

$$\begin{aligned} T(n) &\leq 2(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + \Theta(n) \\ &\leq 2(c(n/2) \lg(n/2)) + \Theta(n) \\ &= cn \lg(n/2) + \Theta(n) \\ &= cn \lg n - cn \lg 2 + \Theta(n) \\ &= cn \lg n - cn + \Theta(n) \\ &\leq cn \lg n. \end{aligned}$$

The last step holds if c, n_0 are sufficiently large that for $n \geq 2n_0$, cn dominates the $\Theta(n)$ term.

Base cases: Need to show that $T(n) \leq cn \lg n$ when $n_0 \leq n < 2n_0$. Add new constraint: $n_0 > 1 \Rightarrow \lg n > 0 \Rightarrow n \lg n > 0$. Pick $n_0 = 2$. Because no base case is given in the recurrence, it's algorithmic $\Rightarrow T(2), T(3)$ are constant. Choose $c = \max \{T(2), T(3)\} \Rightarrow T(2) \leq c < (2 \lg 2)c$ and $T(3) \leq c < (3 \lg 3)c \Rightarrow$ inductive hypothesis established for the base cases.

Wrap up: Have $T(n) \leq cn \lg n$ for all $n \geq 2 \Rightarrow T(n) = O(n \lg n)$.

In practice: Don't usually write out substitution proofs this detailed, especially regarding base cases. For most algorithmic recurrences, the base cases are handled the same way.

Making a good guess

No general way to make a good guess. Experience helps. Can also draw out a recursion tree.

If the recurrence is similar to one you've seen before, try guessing a similar solution. **Example:** $T(n) = 2T(n/2 + 17) + \Theta(n)$. This looks a lot like the merge-sort recurrence $(n) = 2T(n/2) + \Theta(n)$ except for the added 17. When n is large, the difference between $n/2$ and $n/2 + 17$ is small, since both cut n nearly in half. Guess that the solution to the merge-sort recurrence, $T(n) = O(n \lg n)$ works here. (It does.)

When the additive term uses asymptotic notation

- Name the constant in the additive term.
- Show the upper (O) and lower (Ω) bounds separately. Might need to use different constants for each.

[In the book, we don't show how to handle this situation until Section 4.4.]

Example

$T(n) = 2T(n/2) + \Theta(n)$. If we want to show an upper bound of $T(n) = 2T(n/2) + O(n)$, we write $T(n) \leq 2T(n/2) + cn$ for some positive constant c .

Important: We get to name the constant hidden in the asymptotic notation (c in this case), but we do *not* get to choose it, other than assume that it's enough to handle the base case of the recursion.

1. Upper bound:

Guess: $T(n) \leq dn \lg n$ for some positive constant d . This is the inductive hypothesis.

Important: We get to both name and choose the constant in the inductive hypothesis (d in this case). It OK for the constant in the inductive hypothesis (d) to depend on the constant hidden in the asymptotic notation (c).

Substitution:

$$\begin{aligned}
 T(n) &\leq 2T(n/2) + cn \\
 &= 2\left(d\frac{n}{2}\lg\frac{n}{2}\right) + cn \\
 &= dn\lg\frac{n}{2} + cn \\
 &= dn\lg n - dn + cn \\
 &\leq dn\lg n \quad \text{if } -dn + cn \leq 0, \\
 &\quad \quad \quad d \geq c
 \end{aligned}$$

Therefore, $T(n) = O(n \lg n)$.

2. Lower bound: Write $T(n) \geq 2T(n/2) + cn$ for some positive constant c .

Guess: $T(n) \geq dn \lg n$ for some positive constant d .

Substitution:

$$\begin{aligned}
 T(n) &\geq 2T(n/2) + cn \\
 &= 2\left(d\frac{n}{2}\lg\frac{n}{2}\right) + cn \\
 &= dn\lg\frac{n}{2} + cn \\
 &= dn\lg n - dn + cn \\
 &\geq dn\lg n \quad \text{if } -dn + cn \geq 0, \\
 &\quad \quad \quad d \leq c
 \end{aligned}$$

Therefore, $T(n) = \Omega(n \lg n)$.

Therefore, $T(n) = \Theta(n \lg n)$. [For this particular recurrence, we can use $d = c$ for both the upper-bound and lower-bound proofs. That won't always be the case.] ■

Subtracting a low-order term

Might guess the right asymptotic bound, but the math doesn't go through in the proof. Resolve by subtracting a lower-order term.

Example

$T(n) = 2T(n/2) + \Theta(1)$. Guess that $T(n) = O(n)$, and try to show $T(n) \leq cn$ for $n \geq n_0$, where we choose c, n_0 :

$$\begin{aligned} T(n) &\leq 2(c(n/2)) + \Theta(1) \\ &= cn + \Theta(1) . \end{aligned}$$

But this doesn't say that $T(n) \leq cn$ for *any* choice of c .

Could try a larger guess, such as $T(n) = O(n^2)$, but not necessary. We're off only by $\Theta(1)$, a lower-order term. Try subtracting a lower-order term in the guess: $T(n) \leq cn - d$, where $d \geq 0$ is a constant:

$$\begin{aligned} T(n) &\leq 2(c(n/2) - d) + \Theta(1) \\ &= cn - 2d + \Theta(1) \\ &\leq cn - d - (d - \Theta(1)) \\ &\leq cn - d \end{aligned}$$

as long as d is larger than the constant in $\Theta(1)$.

Why subtract off a lower-order term, rather than add it? Notice that it's subtracted twice. Adding a lower-order term twice would take us further away from the inductive hypothesis. Subtracting it twice gives us $T(n) \leq cn - d - (d - \Theta(1))$, and it's easy to choose d to make that inequality hold.

Important: Once again, we get to name and choose the constant c in the inductive hypothesis. And we also get to name and choose the constant d that we subtract off.

Be careful when using asymptotic notation

A false proof for the recurrence $T(n) = 2T(\lfloor n/2 \rfloor) + \Theta(n)$, that $T(n) = O(n)$:

$$\begin{aligned} T(n) &\leq 2 \cdot O(\lfloor n/2 \rfloor) + \Theta(n) \\ &= 2 \cdot O(n) + \Theta(n) \\ &= O(n) . \quad \Leftarrow \text{wrong!} \end{aligned}$$

This "proof" changes the constant in the Θ -notation. Can see this by using an explicit constant. Assume $T(n) \leq cn$ for all $n \geq n_0$:

$$\begin{aligned} T(n) &\leq 2(c \lfloor n/2 \rfloor) + \Theta(n) \\ &\leq cn + \Theta(n) , \end{aligned}$$

but $cn + \Theta(n) > cn$.

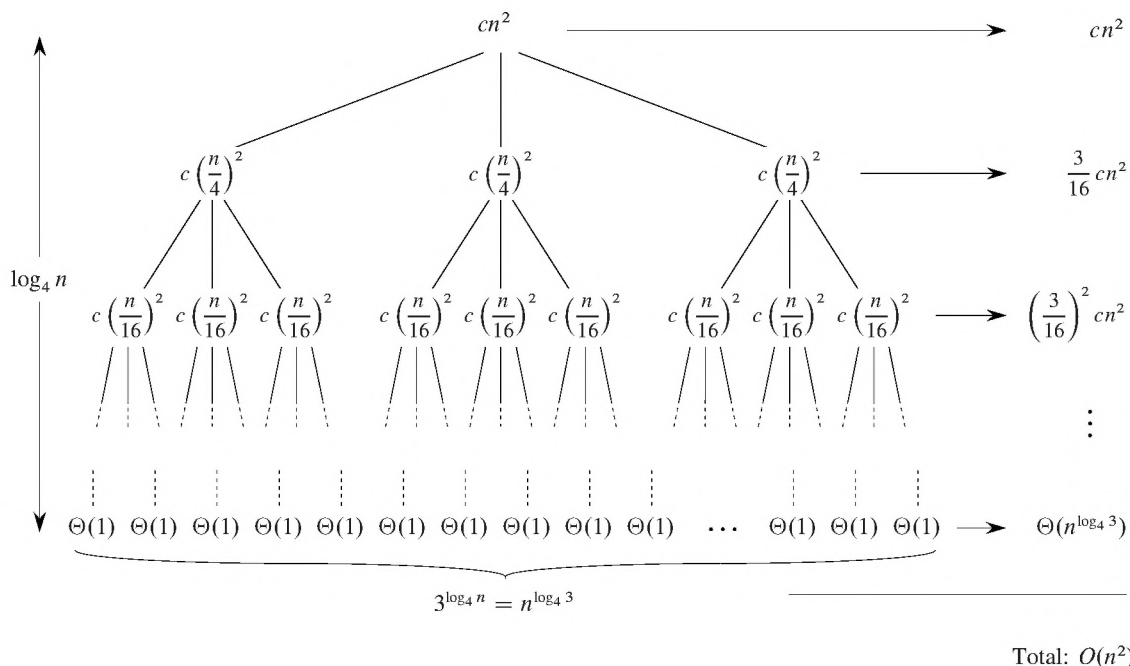
Recursion trees

Use to generate a guess. Then verify by substitution method.

Example

$$T(n) = 3T(n/4) + \Theta(n^2).$$

Draw out a recursion tree for $T(n) = 3T(n/4) + cn^2$:



For simplicity, assume that n is a power of 4 and the base case is $T(1) = \Theta(1)$. Subproblem size for nodes at depth i is $n/4^i$. Get to base case when $n/4^i = 1 \Rightarrow n = 4^i \Rightarrow i = \log_4 n$.

Each level has 3 times as many nodes as the level above, so that depth i has 3^i nodes. Each internal node at depth i has cost $c(n/4^i)^2 \Rightarrow$ total cost at depth i (except for leaves) is $3^i c(n/4^i)^2 = (3/16)^i cn^2$. Bottom level has depth $\log_4 n \Rightarrow$ number of leaves is $3^{\log_4 n} = n^{\log_4 3}$. Since each leaf contributes $\Theta(1)$, total cost of leaves is $\Theta(n^{\log_4 3})$.

Add up costs over all levels to determine cost for the entire tree:

$$\begin{aligned}
 T(n) &= \sum_{i=0}^{\log_4 n} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
 &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
 &= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3}) \\
 &= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\
 &= O(n^2).
 \end{aligned}$$

Idea: Coefficients of cn^2 form a decreasing geometric series. Bound it by an infinite series, and get a bound of 16/13 on the coefficients.

Use substitution method to verify $O(n^2)$ upper bound. Show that $T(n) \leq dn^2$ for constant $d > 0$:

$$\begin{aligned} T(n) &\leq 3T(n/4) + cn^2 \\ &\leq 3d(n/4)^2 + cn^2 \\ &= \frac{3}{16}dn^2 + cn^2 \\ &\leq dn^2, \end{aligned}$$

by choosing $d \geq (16/13)c$. [Again, we get to name but not choose c , and we get to name and choose d .]

That gives an upper bound of $O(n^2)$. The lower bound of $\Omega(n^2)$ is obvious because the recurrence contains a $\Theta(n^2)$ term. Hence, $T(n) = \Theta(n^2)$.

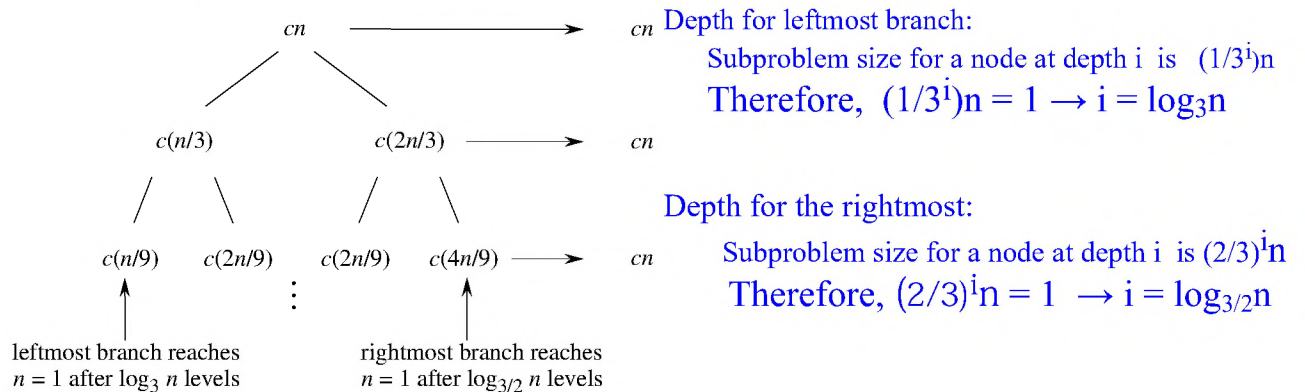
Irregular example

Note: not a complete binary tree.

$$T(n) = T(n/3) + T(2n/3) + \Theta(n).$$

For upper bound, rewrite as $T(n) \leq T(n/3) + T(2n/3) + cn$; for lower bound, as $T(n) \geq T(n/3) + T(2n/3) + cn$.

By summing across each level, the recursion tree shows the cost at each level of recursion (minus the costs of recursive calls, which appear in subtrees):



[This is a simpler way to draw the recursion tree than in Figure 4.2 in the book.]

- There are $\log_3 n$ full levels (going down the left side), and after $\log_{3/2} n$ levels, the problem size is down to 1 (going down the right side).
- Each level contributes $\leq cn$.
- Lower bound guess: $\geq dn \log_3 n = \Omega(n \lg n)$ for some positive constant d .
- Upper bound guess: $\leq dn \log_{3/2} n = O(n \lg n)$ for some positive constant d .
- Then *prove* by substitution.

1. Upper bound:

Guess: $T(n) \leq dn \lg n$.

Substitution:

$$\begin{aligned} T(n) &\leq T(n/3) + T(2n/3) + cn \\ &\leq d(n/3) \lg(n/3) + d(2n/3) \lg(2n/3) + cn \end{aligned}$$

$$\begin{aligned}
&= (d(n/3) \lg n - d(n/3) \lg 3) \\
&\quad + (d(2n/3) \lg n - d(2n/3) \lg(3/2)) + cn \quad \lg(2n/3) = \lg(n/3/2) \\
&= dn \lg n - d((n/3) \lg 3 + (2n/3) \lg(3/2)) + cn \\
&= dn \lg n - d((n/3) \lg 3 + (2n/3) \lg 3 - (2n/3) \lg 2) + cn \\
&= dn \lg n - dn(\lg 3 - 2/3) + cn \\
&\leq dn \lg n \quad \text{if } -dn(\lg 3 - 2/3) + cn \leq 0, \\
&\quad \quad \quad d \geq \frac{c}{\lg 3 - 2/3}.
\end{aligned}$$

Therefore, $T(n) = O(n \lg n)$.

[As before, can name but not choose the constant c hidden in the additive term of $\Theta(n)$. Can both name and choose the constant d in the guess (inductive hypothesis).]

2. Lower bound:

Guess: $T(n) \geq dn \lg n$.

Substitution: Same as for the upper bound, but replacing $\leq$ by $\geq$. End up needing

$$0 < d \leq \frac{c}{\lg 3 - 2/3}.$$

Therefore, $T(n) = \Omega(n \lg n)$.

Since $T(n) = O(n \lg n)$ and $T(n) = \Omega(n \lg n)$, conclude that $T(n) = \Theta(n \lg n)$.

[Omitting the analysis for the number of leaves.]

Master method

Used for many divide-and-conquer **master recurrences** of the form

$$T(n) = aT(n/b) + f(n),$$

where $a \geq 1$, $b > 1$, and $f(n)$ is an asymptotically nonnegative function defined over all sufficiently large positive numbers.

Master recurrences describe recursive algorithms that divide a problem of size n into a subproblems, each of size n/b . Each recursive subproblem takes time $T(n/b)$ (unless it's a base case). Call $f(n)$ the **driving function**.

In reality, subproblem sizes are integers, so that the real recurrence is more like

$$T(n) = a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil) + f(n),$$

where $a', a'' \geq 0$ and $a' + a'' = a$. Ignoring floors and ceilings does not change the asymptotic solution to the recurrence.

Based on the **master theorem** (Theorem 4.1):

Let $a, b, n_0 > 0$ be constants, $f(n)$ be a driving function defined and nonnegative on all sufficiently large reals. Define recurrence $T(n)$ on $n \in \mathbb{N}$ by

$$T(n) = aT(n/b) + f(n),$$

and where $aT(n/b)$ actually means $a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil)$ for some constants $a', a'' \geq 0$ satisfying $a = a' + a''$.

[The mathematics requires only that $a > 0$, but since in practice the number of subproblems is at least 1, the recurrences we see all have $a \geq 1$.]

Then you can solve the recurrence by comparing $n^{\log_b a}$ vs. $f(n)$:

Case 1: $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$.

($f(n)$ is polynomially smaller than $n^{\log_b a}$.)

Solution: $T(n) = \Theta(n^{\log_b a})$.

(Intuitively: cost is dominated by leaves.)

F(n) is polynomially smaller than $n^{\log_b a}$

Case 2: $f(n) = \Theta(n^{\log_b a} \lg^k n)$, where $k \geq 0$ is a constant.

($f(n)$ is within a polylog factor of $n^{\log_b a}$, but not smaller.)

Solution: $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

(Intuitively: cost is $n^{\log_b a} \lg^k n$ at each level, and there are $\Theta(\lg n)$ levels.)

Simple case: $k = 0 \Rightarrow f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \lg n)$.

F(n) is the same size as $n^{\log_b a}$

Case 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$ and $f(n)$ satisfies the regularity condition $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n .

($f(n)$ is polynomially greater than $n^{\log_b a}$.)

Solution: $T(n) = \Theta(f(n))$.

(Intuitively: cost is dominated by root.)

F(n) is polynomially larger than $n^{\log_b a}$. It should also satisfy the regularity condition

What's with the Case 3 regularity condition?

- Generally not a problem.
- It always holds whenever $f(n) = n^k$ and $f(n) = \Omega(n^{\log_b a + \epsilon})$ for constant $\epsilon > 0$. [Proving this makes a nice homework exercise. See below.] So you don't need to check it when $f(n)$ is a polynomial.

[Here's a proof that the regularity condition holds when $f(n) = n^k$ and $f(n) = \Omega(n^{\log_b a + \epsilon})$ for constant $\epsilon > 0$.

Since $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $f(n) = n^k$, we have that $k > \log_b a$. Using a base of b and treating both sides as exponents, we have $b^k > b^{\log_b a} = a$, and so $a/b^k < 1$. Since a, b , and k are constants, if we let $c = a/b^k$, then c is a constant strictly less than 1. We have that $af(n/b) = a(n/b)^k = (a/b^k)n^k = cf(n)$, and so the regularity condition is satisfied.]

Call $n^{\log_b a}$ the **watershed function**. Master method compares the driving function $f(n)$ with the watershed function $n^{\log_b a}$.

- If the watershed function grows *polynomially faster* than the driving function, then case 1 applies. Example (not likely to see in algorithm analysis): $T(n) = 4T(n/2) + n^{1.99}$. Watershed function is $n^{\log_2 4} = n^2$, which is polynomially larger than $n^{1.99}$ by a factor of $n^{0.01}$. Case 1 would apply $\Rightarrow T(n) = \Theta(n^2)$.
- If the driving function grows *polynomially faster* than the watershed function and the regularity condition holds, then case 3 applies. Example: $T(n) = 4T(n/2) + n^{2.01}$. Now the driving function is polynomially larger than the watershed function by a factor of $n^{0.01}$. Case 3 would apply $\Rightarrow T(n) = \Theta(n^{2.01})$.

- There are gaps between cases 1 and 2 and between cases 2 and 3. Example: $T(n) = 2T(n/2) + n/\lg n \Rightarrow$ watershed function is $n^{\lg_2 2} = n$ and driving function is $f(n) = n/\lg n$. Have $f(n) = o(n)$, so that $f(n)$ grows more slowly than n , it doesn't grow *polynomially slower*. In terms of the master theorem, have $f(n) = n \lg^{-1} n$, so that $k = -1$. Master theorem holds only for $k \geq 0$, so case 2 does not apply.

Examples [different from those in the book]

- $T(n) = 5T(n/2) + \Theta(n^2)$
 $n^{\lg_2 5}$ vs. n^2
 Since $\lg_2 5 - \epsilon = 2$ for some constant $\epsilon > 0$, use case 1 $\Rightarrow T(n) = \Theta(n^{\lg 5})$
- $T(n) = 27T(n/3) + \Theta(n^3 \lg n)$
 $n^{\lg_3 27} = n^3$ vs. $n^3 \lg n$
 Use case 2 with $k = 1 \Rightarrow T(n) = \Theta(n^3 \lg^2 n)$
- $T(n) = 5T(n/2) + \Theta(n^3)$
 $n^{\lg_2 5}$ vs. n^3
 Now $\lg 5 + \epsilon = 3$ for some constant $\epsilon > 0$
 Check regularity condition (don't really need to since $f(n)$ is a polynomial):
 $af(n/b) = 5(n/2)^3 = 5n^3/8 \leq cn^3$ for $c = 5/8 < 1$
 Use case 3 $\Rightarrow T(n) = \Theta(n^3)$
- $T(n) = 27T(n/3) + \Theta(n^3/\lg n)$
 $n^{\lg_3 27} = n^3$ vs. $n^3/\lg n = n^3 \lg^{-1} n \neq \Theta(n^3 \lg^k n)$ for any $k \geq 0$.
Cannot use the master method.